ESPECIFICAÇÃO DE ARQUITETURA & ENGENHARIA DE SOFTWARE

CoreLedger: Motor Distribuído de Pagamentos e Conciliação Contábil

Requisitos de Sistema, Matriz Tecnológica Java/Spring Boot, Critérios de Aceite e Guia de Implementação

1. Visão Geral do Produto e Contexto de Negócio

O **CoreLedger** é uma plataforma de backend projetada em Java 21 e Spring Boot 3.3+ para processar transações financeiras críticas, liquidações interbancárias e conciliação contábil pelo método das partidas dobradas (*double-entry bookkeeping*). O sistema resolve os desafios clássicos de sistemas financeiros distribuídos: prevenção estrita contra gasto duplo (*double-spending*), eliminação de condições de corrida em saldos concorrentes, comunicação assíncrona resiliente e rastreabilidade integral para auditoria.

A arquitetura adota o padrão de microsserviços orientados a eventos com garantias transacionais fortes no núcleo relacional e consistência eventual controlada na integração com serviços periféricos e adquirentes externas. Todas as operações de mutação financeira exigem autenticação OAuth2/JWT, chave de idempotência exclusiva e registro simultâneo na tabela de eventos de saída (*Transactional Outbox Pattern*).

> **Invariante Contábil Fundamental:** Nenhuma alteração de saldo pode ocorrer por atualização direta isolada. Toda movimentação financeira exige a gravação atômica de pelo menos dois lançamentos no livro-razão (um registro de `DEBIT` na conta de origem e um registro de `CREDIT` de igual valor na conta de destino), mantendo a soma global do sistema rigorosamente igual a zero.

2. Matriz Tecnológica e Bibliotecas Vinculadas

A tabela a seguir relaciona todas as bibliotecas oficiais do ecossistema Spring e dependências complementares necessárias para cumprir os requisitos de segurança, observabilidade, resiliência, documentação e persistência.

Domínio Técnico	Artefato Maven (Dependência)	Papel na Arquitetura CoreLedger
Núcleo Web	`spring-boot-starter-web` `spring-boot-starter-validation`	Exposição de APIs RESTful com suporte a Virtual Threads (Java 21) e validação declarativa de payloads via JSR-380 (Bean Validation).
Segurança e Autenticação	`spring-boot-starter-security` `spring-boot-starter-oauth2-resource-server`	Validação stateless de tokens JWT assinados com RSA (JWKS), controle de acesso baseado em papéis (RBAC), escopos OAuth2 e proteção de rotas.

Domínio Técnico	Artefato Maven (Dependência)	Papel na Arquitetura CoreLedger
Documentação OpenAPI	`springdoc-openapi-starter-webmvc-ui`	Geração automatizada da especificação OpenAPI 3.0 e interface interativa Swagger UI com suporte nativo a autenticação Bearer JWT.
Observabilidade e Telemetria	`spring-boot-starter-actuator` `micrometer-registry-prometheus` `micrometer-tracing-bridge-otel` `opentelemetry-exporter-otlp`	Sondas de saúde (liveness/readiness), coleta de métricas de negócio e infraestrutura para o Prometheus e rastreamento distribuído (W3C TraceContext).
Tolerância a Falhas	`resilience4j-spring-boot3` `resilience4j-micrometer`	Implementação de Circuit Breaker, Rate Limiter, Bulkhead e Retry com recuo exponencial nas integrações síncronas com adquirentes.
Persistência e Migração	`spring-boot-starter-data-jpa` `postgresql` `flyway-core`	Acesso transacional ACID com bloqueio otimista/pessimista no PostgreSQL e versionamento estrito de esquema de banco de dados via Flyway.
Cache e Idempotência	`spring-boot-starter-data-redis` `redisson-spring-boot-starter`	Armazenamento de chaves de idempotência com tempo de expiração (TTL) e travas distribuídas (*distributed locks*) para contas sob alta concorrência.
Mensageria Assíncrona	`spring-boot-starter-amqp`	Publicação e consumo de eventos financeiros no RabbitMQ com confirmação de entrega (*Publisher Confirms*) e roteamento para Dead Letter Queues (DLQ).
Qualidade e Testes	`spring-boot-starter-test` `testcontainers` (PostgreSQL, Redis, RabbitMQ)	Testes unitários (JUnit 5 + Mockito), testes de fatia (`@WebMvcTest` , `@DataJpaTest`) e suíte de integração com infraestrutura efêmera real.

3. Especificação de Requisitos

3.1 Requisitos Funcionais (RF)

Código	Requisito	Especificação Detalhada
RF-01	Gestão de Contas e Saldos	Permitir a criação de contas contábeis identificadas por UUID e número único, armazenando valores exclusivamente em centavos (`BIGINT`) para evitar erros de arredondamento de ponto flutuante.
RF-02	Liquidação por Partidas Dobradas	Processar transferências internas e externas gerando obrigatoriamente um lançamento de débito e um lançamento de crédito vinculados ao mesmo identificador de transação.

Código	Requisito	Especificação Detalhada
RF-03	Controle de Idempotência	Exigir o cabeçalho HTTP `Idempotency-Key` em todas as operações de escrita, garantindo que reenvios de rede nunca dupliquem débitos ou créditos.
RF-04	Integração com Adquirente	Comunicar-se com gateways externos de liquidação aplicando políticas de resiliência e tratamento de indisponibilidade temporária.
RF-05	Publicação via Outbox	Persistir eventos de domínio na tabela `outbox_events` na mesma transação do banco de dados e despachá-los de forma assíncrona ao RabbitMQ.
RF-06	Extrato e Conciliação	Disponibilizar consulta paginada do razão contábil por período e endpoint de verificação de integridade entre o saldo consolidado e a soma das entradas históricas.

3.2 Requisitos Não Funcionais (RNF)

Código	Categoria	Regra de Engenharia e Restrição Técnica
RNF-01	Autenticação e Autorização	Arquitetura OAuth2 Resource Server stateless. Endpoints de escrita exigem escopo `SCOPE_payment:write` e endpoints de leitura exigem `SCOPE_ledger:read`. Erros de acesso devem respeitar o formato RFC 7807 (Problem Details).
RNF-02	Observabilidade e Rastreamento	Exposição de métricas no endpoint `/actuator/prometheus`. Logs estruturados em JSON com injeção automática de `traceId` e `spanId` no MDC do SLF4J/Logback.
RNF-03	Documentação Interativa	Interface Swagger UI acessível em `/swagger-ui.html` documentando todos os contratos DTO, regras de validação, exemplos de payload e códigos de retorno HTTP (201, 400, 401, 403, 409, 422, 503).
RNF-04	Segurança em Concorrência	Uso combinado de bloqueio distribuído no Redis e controle de versão otimista (`@Version`) no JPA para impedir saldo negativo quando múltiplas threads debitam a mesma conta simultaneamente.
RNF-05	Resiliência de Rede	Abertura automática do Circuit Breaker quando a taxa de falha da adquirente atingir 50% em uma janela de 20 chamadas, retornando resposta degradada controlada em menos de 50ms.

4. Critérios de Aceite e Cenários de Homologação

A aprovação técnica da implementação depende do cumprimento integral dos seguintes critérios verificáveis em suíte automatizada de testes de integração:

ID / Escopo	Cenário de Teste (Dado / Quando)	Resultado Esperado (Critério de Aceite)
CA-01 Partidas Dobradas	**Dado** uma conta A com saldo de R$ 500,00 e conta B com R$ 100,00; **Quando** for solicitada uma transferência válida de R$ 150,00 de A para B.	O sistema retorna `201 Created`, atualiza o saldo de A para R$ 350,00 e B para R$ 250,00, grava exatamente 2 registros em `ledger_entries` (-15000 e +15000 centavos) e 1 evento `PENDING` em `outbox_events`.
CA-02 Idempotência Estrita	**Dado** uma requisição enviada com `Idempotency-Key: K-100`; **Quando** uma segunda requisição com a mesma chave `K-100` chegar durante ou após o processamento da primeira.	Se a primeira ainda estiver em execução, retorna `409 Conflict`. Se já tiver sido concluída, retorna `422 Unprocessable Entity` (ou o payload original cacheado) sem gerar novos lançamentos no banco.
CA-03 Concorrência Extrema	**Dado** uma conta com saldo de R$ 100,00; **Quando** 50 threads simultâneas dispararem débitos concorrentes de R$ 20,00 com chaves de idempotência distintas.	Exatamente 5 requisições retornam `201 Created` e 45 requisições retornam `422 Unprocessable Entity` por saldo insuficiente. O saldo final da conta deve ser exatamente R$ 0,00.
CA-04 Segurança OAuth2	**Dado** um cliente sem token JWT ou com token contendo apenas o escopo `SCOPE_ledger:read`; **Quando** tentar executar um `POST /api/v1/transactions`.	Requisições sem token válido recebem `401 Unauthorized`; requisições com escopo insuficiente recebem `403 Forbidden`, sem iniciar nenhuma transação no serviço de domínio.
CA-05 Circuit Breaker	**Dado** que o serviço externo da adquirente apresenta timeout superior a 2 segundos em mais de 50% das últimas 20 chamadas; **Quando** uma nova transação externa for submetida.	O Circuit Breaker transita para o estado `OPEN`, interrompe chamadas de rede imediatas, incrementa a métrica de fallback no Micrometer e preserva o pool de threads da aplicação.
CA-06 Observabilidade	**Dado** uma transação processada (com sucesso ou falha); **Quando** os endpoints do Actuator e os logs da aplicação forem inspecionados.	O contador `ledger_transactions_total` é incrementado no Prometheus com as tags correspondentes e os logs exibem o `traceId` propagado desde a camada HTTP até o consumidor RabbitMQ.

5. Modelagem de Dados e Migração Flyway

O script abaixo deve ser salvo em `src/main/resources/db/migration/V1__init_coreledger_schema.sql`. O versionamento é gerenciado automaticamente pelo Flyway durante a inicialização da aplicação.

```
CREATE TABLE accounts (
    id UUID PRIMARY KEY,
    holder_name VARCHAR(150) NOT NULL,
    account_number VARCHAR(30) UNIQUE NOT NULL,
    balance_cents BIGINT NOT NULL DEFAULT 0 CHECK (balance_cents >= 0),
    version BIGINT NOT NULL DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE transactions (
    id UUID PRIMARY KEY,
    idempotency_key VARCHAR(120) UNIQUE NOT NULL,
    amount_cents BIGINT NOT NULL CHECK (amount_cents > 0),
    status VARCHAR(30) NOT NULL,
    description VARCHAR(255),
    trace_id VARCHAR(64),
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE ledger_entries (
    id UUID PRIMARY KEY,
    transaction_id UUID NOT NULL REFERENCES transactions(id),
    account_id UUID NOT NULL REFERENCES accounts(id),
    entry_type VARCHAR(10) NOT NULL CHECK (entry_type IN ('DEBIT', 'CREDIT')),
    amount_cents BIGINT NOT NULL CHECK (amount_cents > 0),
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE outbox_events (
    id UUID PRIMARY KEY,
    aggregate_type VARCHAR(50) NOT NULL,
    aggregate_id VARCHAR(64) NOT NULL,
    event_type VARCHAR(80) NOT NULL,
    payload JSONB NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_at TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_ledger_tx ON ledger_entries(transaction_id);
CREATE INDEX idx_ledger_acc ON ledger_entries(account_id, created_at DESC);
CREATE INDEX idx_outbox_polling ON outbox_events(status, created_at) WHERE status
= 'PENDING';
```

6. Guia de Implementação Passo a Passo

Passo 1: Configuração de Dependências no Maven (pom.xml)

Configure o projeto com Java 21 e inclua os starters essenciais de segurança, documentação, telemetria, resiliência e banco de dados.

```
<properties>
    <java.version>21</java.version>
    <springdoc.version>2.6.0</springdoc.version>
    <resilience4j.version>2.2.0</resilience4j.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-redis</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-tracing-bridge-otel</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>${springdoc.version}</version>
    </dependency>
    <dependency>
        <groupId>io.github.resilience4j</groupId>
        <artifactId>resilience4j-spring-boot3</artifactId>
        <version>${resilience4j.version}</version>
    </dependency>
    <dependency>
        <groupId>org.flywaydb</groupId>
```

Passo 2: Configuração de Segurança Stateless e Escopos JWT

Implemente a classe `SecurityConfiguration` para liberar apenas rotas públicas de documentação e sondas de infraestrutura, exigindo escopos granulares nos endpoints de negócio.

```
package com.coreledger.infrastructure.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSe
curity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity
;
import org.springframework.security.config.http.SessionCreationPolicy;
import
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticati
onConverter;
import
org.springframework.security.oauth2.server.resource.authentication.JwtGrantedAutho
ritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfiguration {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sm ->
sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**",
"/actuator/prometheus").permitAll()
                .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/swagger-
ui.html").permitAll()
                .requestMatchers(HttpMethod.POST,
"/api/v1/transactions/**").hasAuthority("SCOPE_payment:write")
                .requestMatchers(HttpMethod.GET,
"/api/v1/ledger/**").hasAuthority("SCOPE_ledger:read")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt ->
jwt.jwtAuthenticationConverter(jwtAuthenticationConverter()))
            )
            .build();
    }

    private JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtGrantedAuthoritiesConverter scopesConverter = new
JwtGrantedAuthoritiesConverter();
        scopesConverter.setAuthorityPrefix("SCOPE_");
```

Passo 3: Configuração do Swagger / OpenAPI 3 com Esquema JWT

Configure o bean `OpenAPI` para permitir que desenvolvedores e avaliadores testem endpoints autenticados diretamente pelo navegador através do botão *Authorize*.

```
package com.coreledger.infrastructure.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI coreLedgerOpenAPI() {
        final String schemeName = "BearerAuth";
        return new OpenAPI()
            .info(new Info()
                .title("CoreLedger API - Motor de Pagamentos e Conciliação")
                .version("1.0.0")
                .description("Documentação interativa de endpoints transacionais,
auditoria e saldos."))
            .addSecurityItem(new SecurityRequirement().addList(schemeName))
            .components(new Components()
                .addSecuritySchemes(schemeName, new SecurityScheme()
                    .name(schemeName)
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT")));
    }
}
```

Passo 4: Aspecto de Idempotência com Redis

Intercepte todas as chamadas aos controllers transacionais validando o cabeçalho `Idempotency-Key` de forma atômica no Redis via comando `SETNX` (`setIfAbsent`).

```
package com.coreledger.infrastructure.idempotency;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ResponseStatusException;
import java.time.Duration;

@Aspect
@Component
public class IdempotencyAspect {

    private final StringRedisTemplate redisTemplate;
    private static final Duration TTL = Duration.ofHours(24);

    public IdempotencyAspect(StringRedisTemplate redisTemplate) {
        this.redisTemplate = redisTemplate;
    }

    @Around("@annotation(IdempotentEndpoint) && args(idempotencyKey, ..)")
    public Object checkIdempotency(ProceedingJoinPoint pjp, String idempotencyKey)
throws Throwable {
        if (idempotencyKey == null || idempotencyKey.isBlank()) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Cabeçalho
Idempotency-Key ausente.");
        }

        String redisKey = "coreledger:idemp:" + idempotencyKey;
        Boolean locked = redisTemplate.opsForValue().setIfAbsent(redisKey,
"IN_PROGRESS", TTL);

        if (Boolean.FALSE.equals(locked)) {
            String status = redisTemplate.opsForValue().get(redisKey);
            if ("IN_PROGRESS".equals(status)) {
                throw new ResponseStatusException(HttpStatus.CONFLICT, "Transação
já está em processamento.");
            }
            throw new ResponseStatusException(HttpStatus.UNPROCESSABLE_ENTITY,
"Chave de idempotência já utilizada.");
        }

        try {
            Object response = pjp.proceed();
            redisTemplate.opsForValue().set(redisKey, "COMPLETED", TTL);
            return response;
        } catch (Exception ex) {
            redisTemplate.delete(redisKey);
            throw ex;
        }
    }
}
```

Passo 5: Serviço de Domínio com Partidas Dobradas e Métricas

O serviço central executa a transferência em transação ACID, registra as partidas dobradas, grava o evento no Outbox e alimenta os coletores do Micrometer.

```
package com.coreledger.domain.service;

import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.UUID;

@Service
public class LedgerTransferService {

    private final AccountRepository accountRepository;
    private final TransactionRepository transactionRepository;
    private final LedgerEntryRepository entryRepository;
    private final OutboxRepository outboxRepository;
    private final MeterRegistry meterRegistry;
    private final Timer transferTimer;

    public LedgerTransferService(
            AccountRepository accountRepository,
            TransactionRepository transactionRepository,
            LedgerEntryRepository entryRepository,
            OutboxRepository outboxRepository,
            MeterRegistry meterRegistry) {
        this.accountRepository = accountRepository;
        this.transactionRepository = transactionRepository;
        this.entryRepository = entryRepository;
        this.outboxRepository = outboxRepository;
        this.meterRegistry = meterRegistry;
        this.transferTimer = Timer.builder("ledger.transfer.latency")
            .publishPercentiles(0.5, 0.95, 0.99)
            .register(meterRegistry);
    }

    @Transactional
    public UUID executeTransfer(String idempotencyKey, UUID sourceId, UUID
targetId, long amountCents) {
        return transferTimer.record(() -> {
            Account source = accountRepository.findByIdWithLock(sourceId)
                .orElseThrow(() -> new IllegalArgumentException("Conta de origem
não encontrada"));
            Account target = accountRepository.findByIdWithLock(targetId)
                .orElseThrow(() -> new IllegalArgumentException("Conta de destino
não encontrada"));

            if (source.getBalanceCents() < amountCents) {
                meterRegistry.counter("ledger.transactions.total", "status",
"INSUFFICIENT_FUNDS").increment();
                throw new InsufficientBalanceException("Saldo insuficiente para a
operação.");
            }

            source.debit(amountCents);
            target.credit(amountCents);
```

Passo 6: Parametrização de Observabilidade e Resiliência (application.yml)

Configure o arquivo `src/main/resources/application.yml` para habilitar Virtual Threads, rastreamento OTel, exposição do Prometheus e limites do Resilience4j.

```yaml
spring:
  application:
    name: coreledger-engine
  threads:
    virtual:
      enabled: true
  datasource:
    url: jdbc:postgresql://localhost:5432/coreledger
    username: ${DB_USER:postgres}
    password: ${DB_PASS:postgres}
    hikari:
      maximum-pool-size: 30
      connection-timeout: 5000
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      show-details: always
      probes:
        enabled: true
  tracing:
    sampling:
      probability: 1.0
  metrics:
    tags:
      application: ${spring.application.name}

resilience4j:
  circuitbreaker:
    instances:
      acquirerGateway:
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 15s
        permittedNumberOfCallsInHalfOpenState: 5
  retry:
    instances:
      acquirerGateway:
        maxAttempts: 3
        waitDuration: 400ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
```

7. Estratégia de Testes e Checklist de Prontidão

Para consolidar o projeto como referência de engenharia em portfólio e entrevistas técnicas, estruture a pirâmide de testes em três camadas complementares:

- **Testes de Unidade de Domínio (JUnit 5 + AssertJ):** Validação isolada das regras de débito, crédito, invariante de soma zero e bloqueio de quantias negativas ou zeradas em menos de 50ms.
- **Testes de Fatia de API (`@WebMvcTest` + `spring-security-test`):** Simulação de chamadas HTTP com `MockMvc` injetando tokens JWT simulados (`jwt().authorities(...)`) para verificar os retornos `401` , `403` , `400` e `201` .
- **Testes de Concorrência Real com Testcontainers:** Inicialização de containers reais do PostgreSQL e Redis via anotação `@ServiceConnection` do Spring Boot 3.1+, disparando um `ExecutorService` com 50 threads simultâneas contra o endpoint de transferência para comprovar a ausência de condições de corrida e a precisão do saldo final.